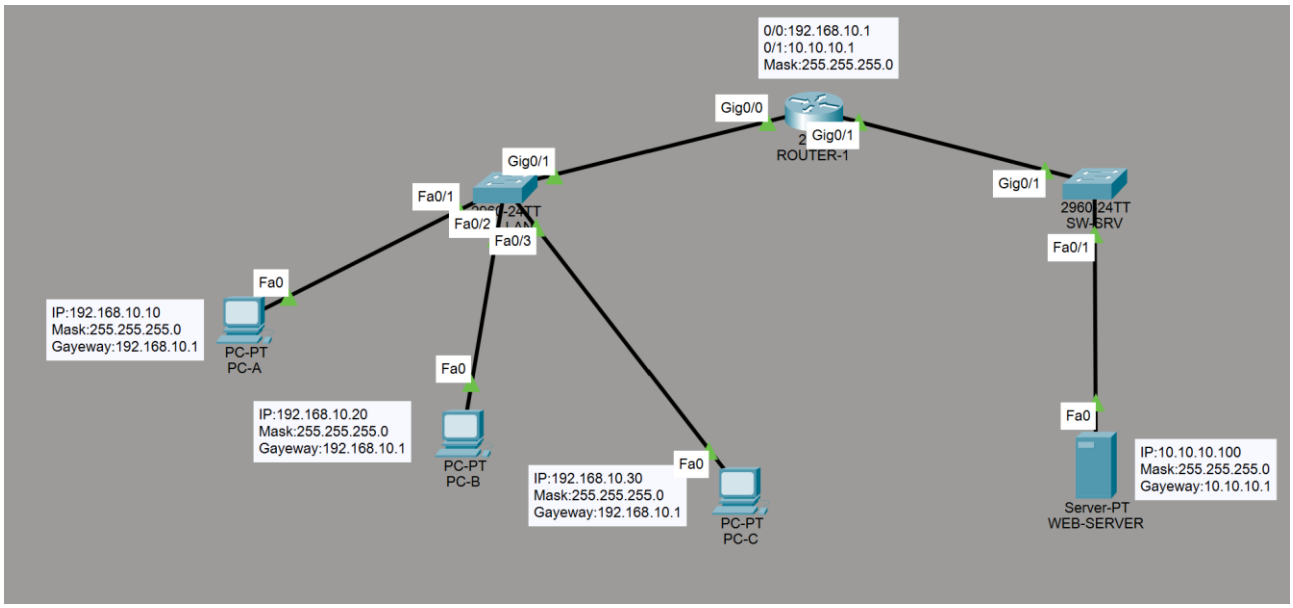
Lab 08: ICMP Traffic Filtering with Cisco ACLs

Phase 1 — Build the Topolog

☑ Phase 1 Checkpoint:

Full Packet Tracer canvas with all devices placed, named, connected, and interfaces up:



Output of 'show ip interface brief' on ROUTER-1 showing both interfaces as 'up / up':



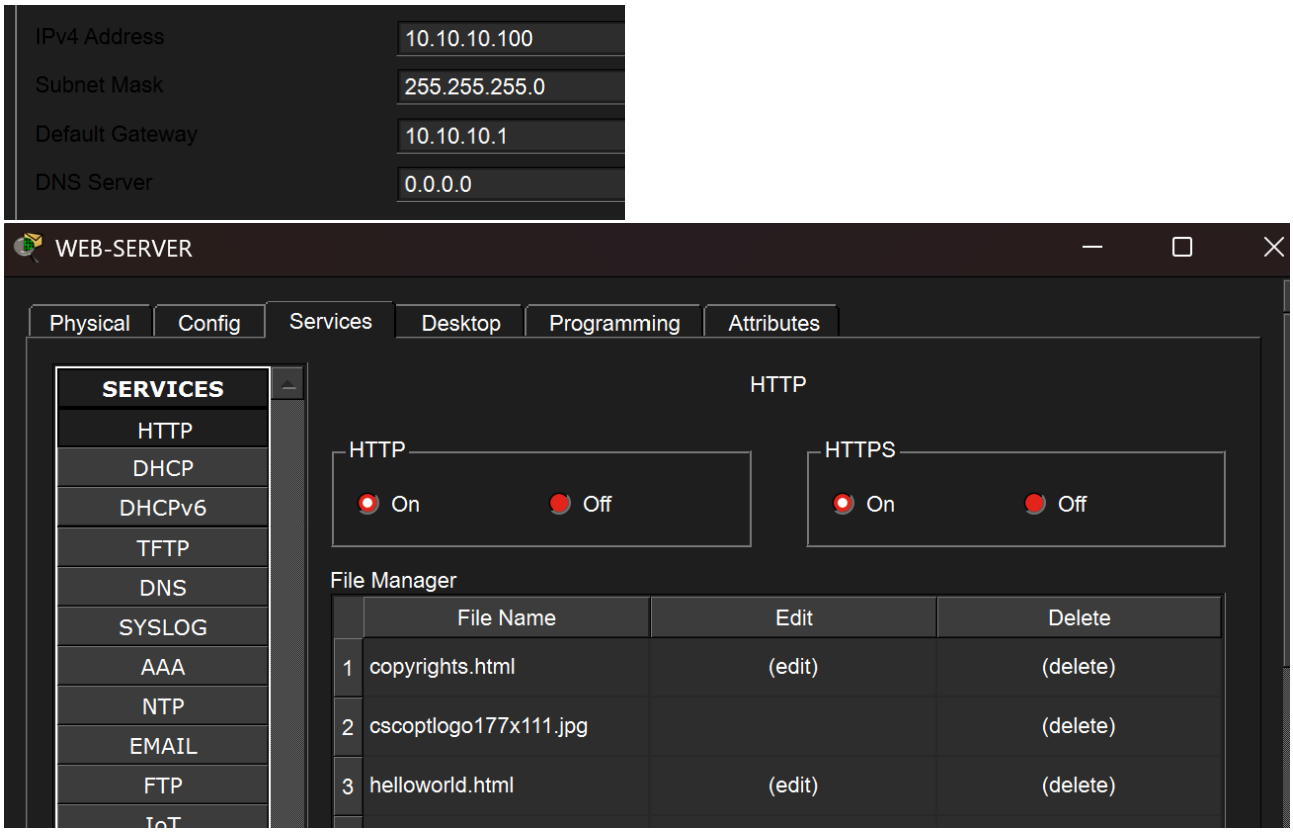
IP configuration of each PC (Desktop → IP Configuration) :







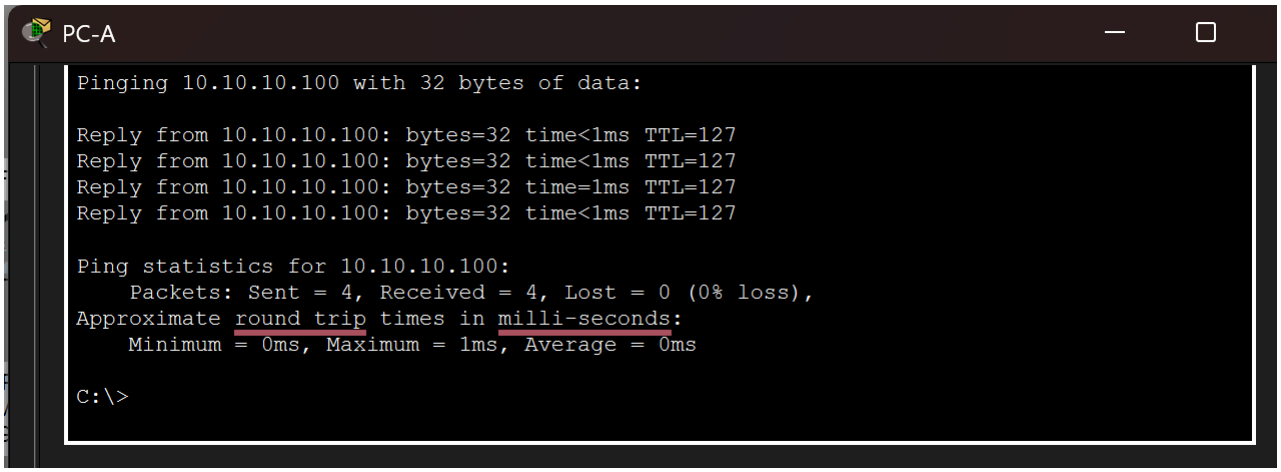
IP configuration of WEB-SERVER and the HTTP service turned ON:



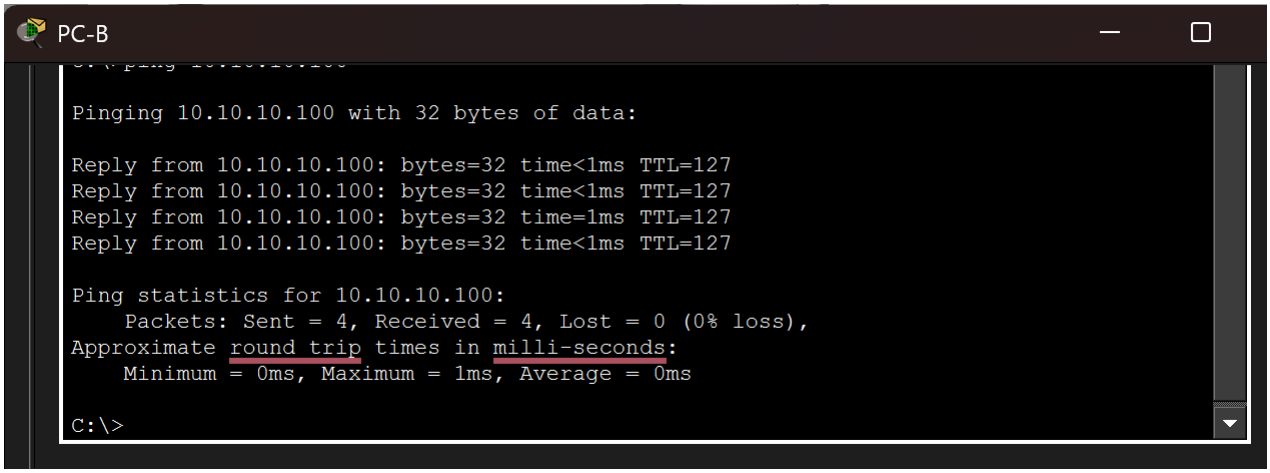
Phase 2 — Baseline Test (Before ACL)

☑ Phase 2 Checkpoint

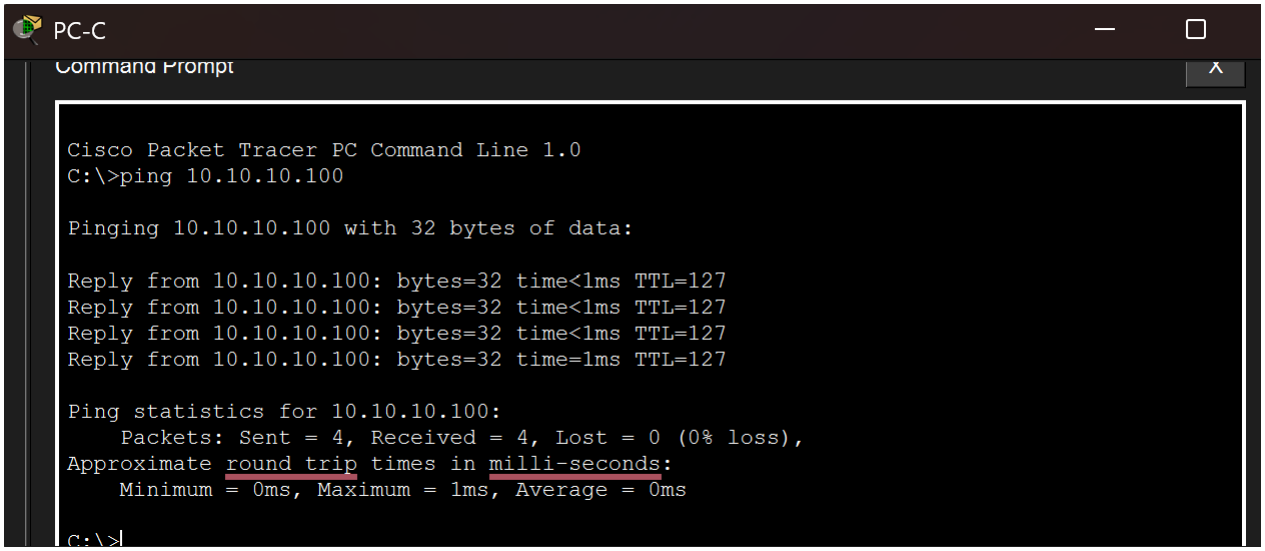
Ping from PC-A to WEB-SERVER — successful :



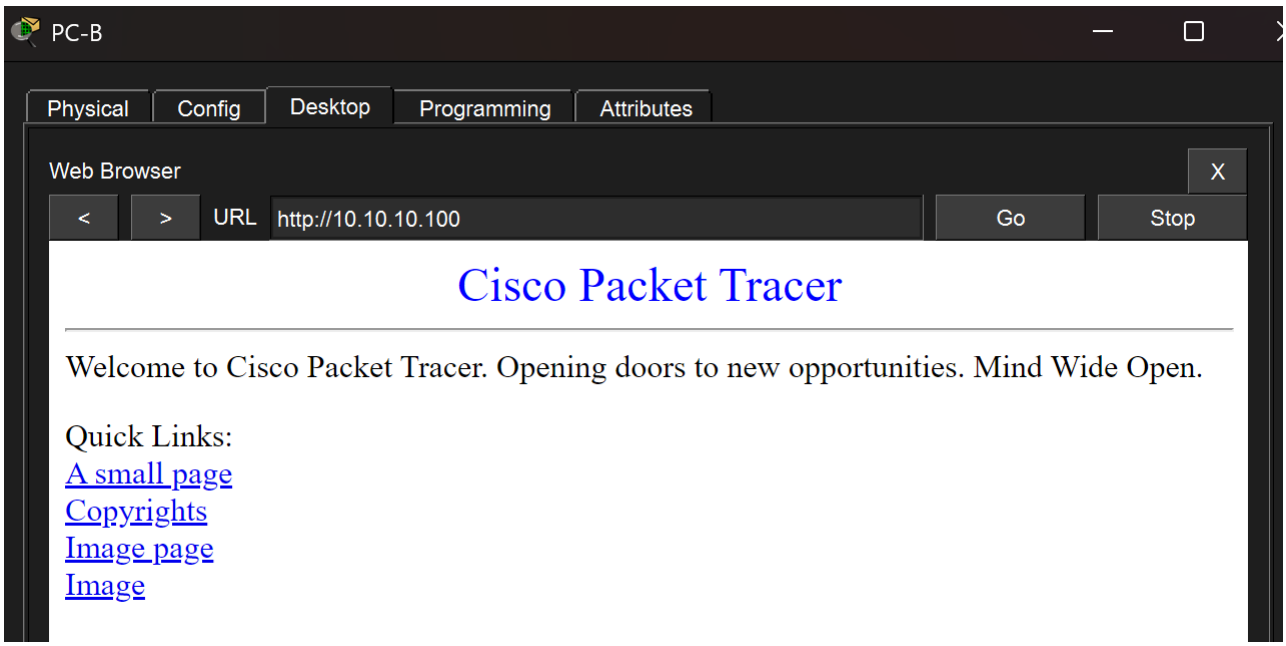
Ping from PC-B to WEB-SERVER — successful :



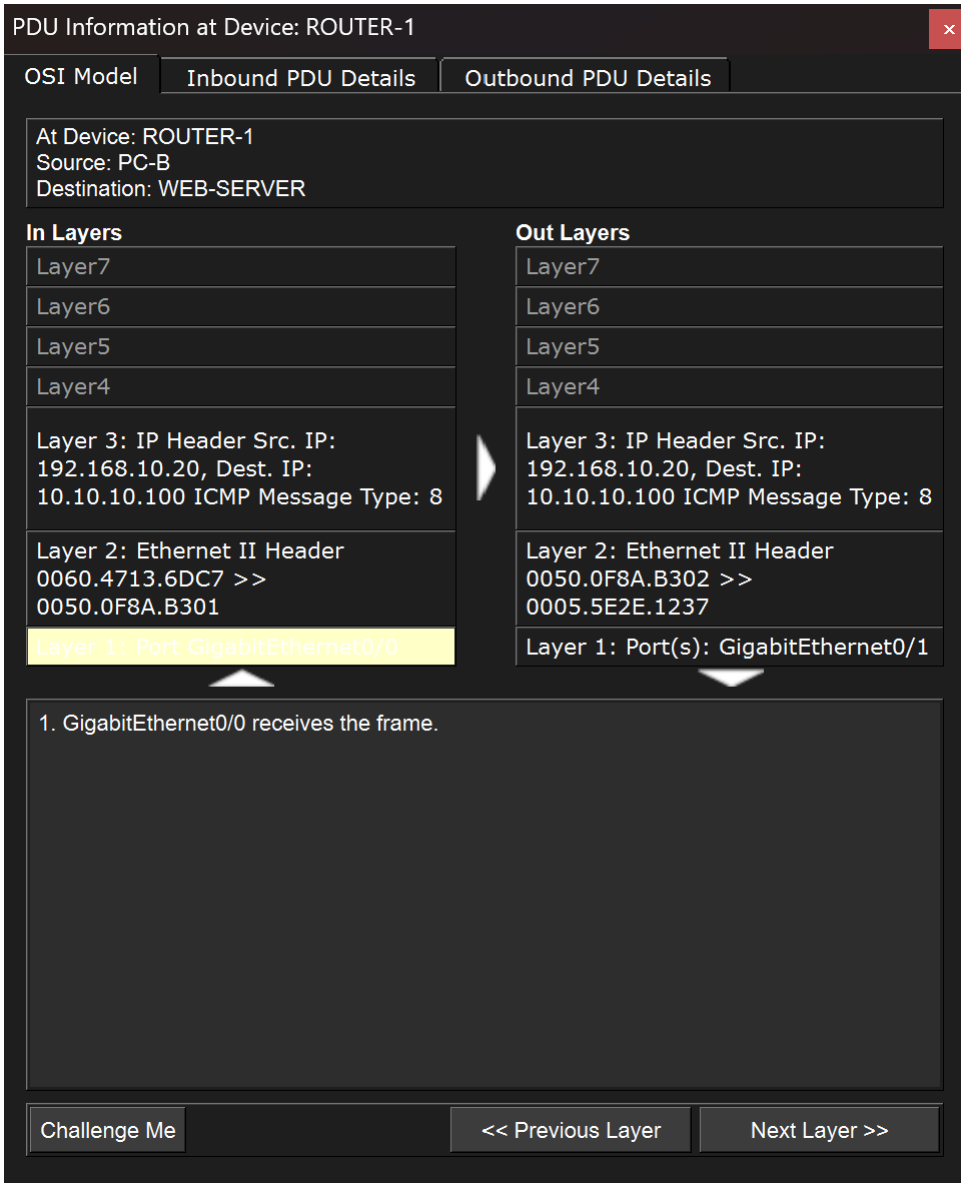
Ping from PC-C to WEB-SERVER — successful:



Web browser from PC-B showing the HTTP page loading from 10.10.10.100:



Simulation Mode PDU capture showing ICMP packet forwarded from PC-B through ROUTER-1.



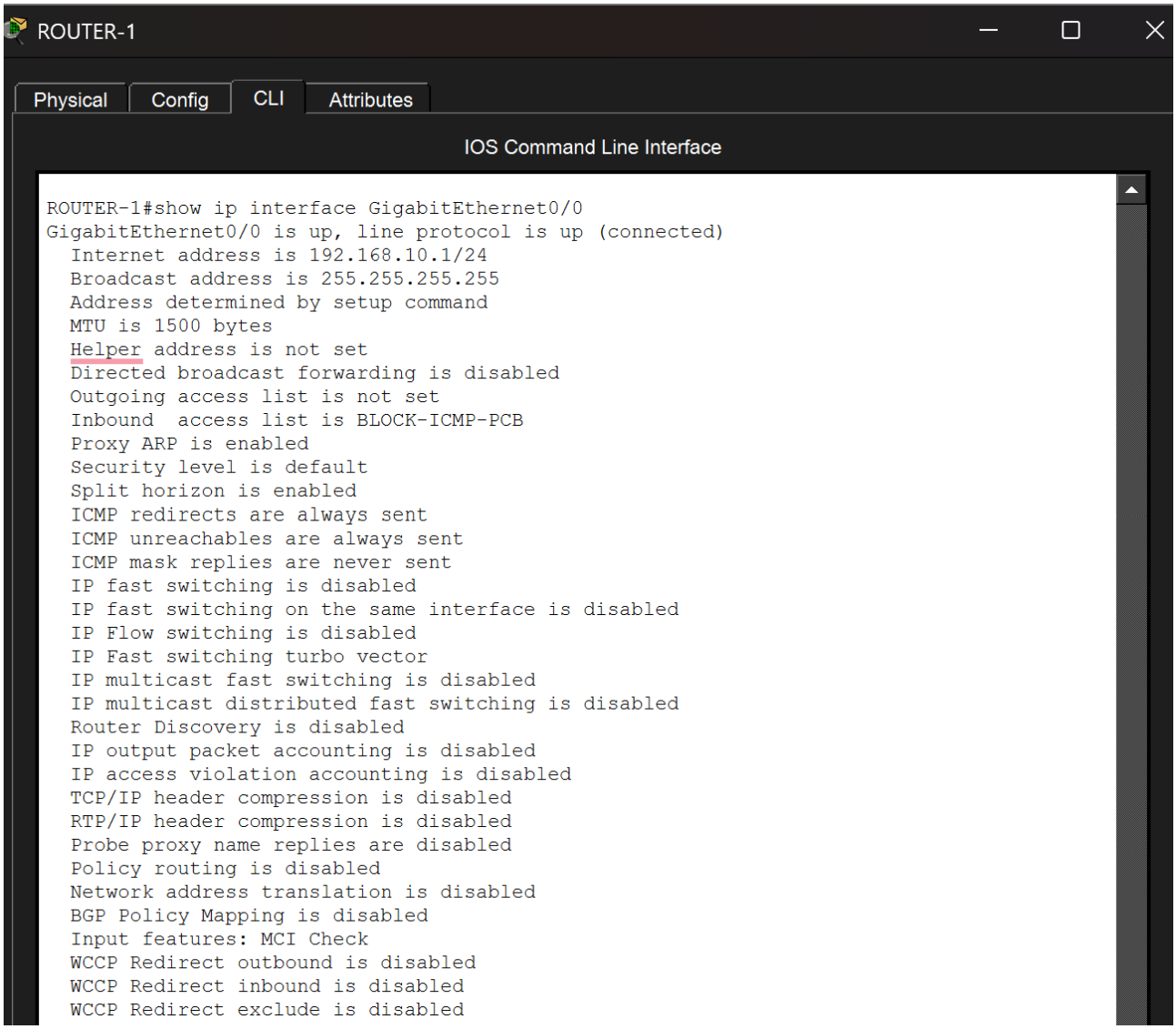
Phase 3 — Configure the ACL on ROUTER-1

☑ Phase 3 Checkpoint

Full ACL configuration from 'show access-lists' output:



'show ip interface GigabitEthernet0/0' output confirming the ACL is applied inbound:



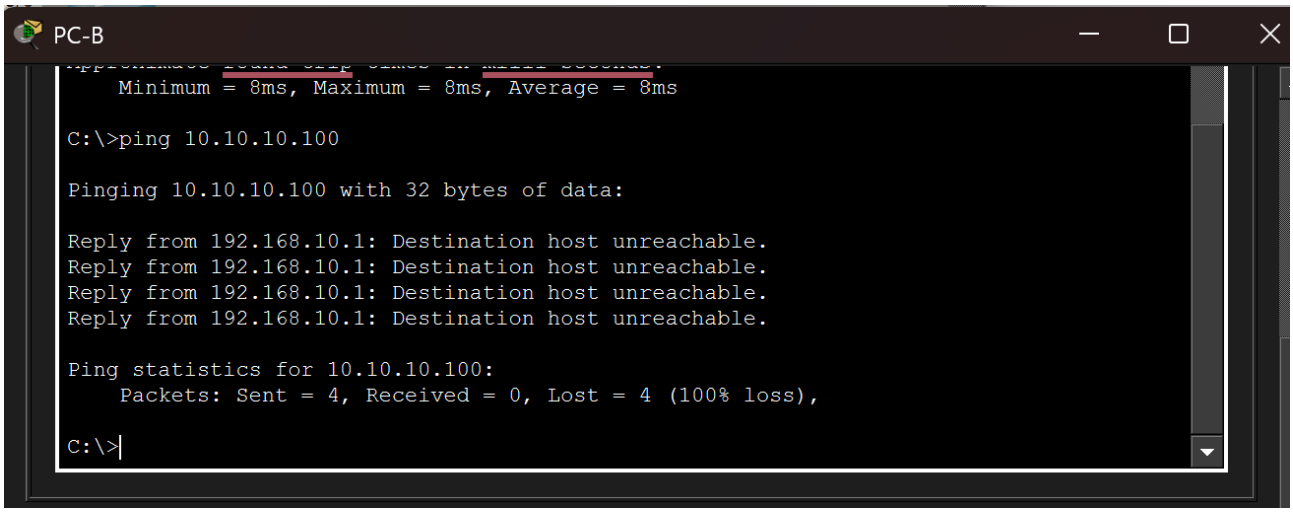
ROUTER-1 running config showing both the ACL definition and the interface assignment (use 'show running-config' and scroll to the relevant sections):

```
!
interface GigabitEthernet0/0
 description Gateway for LAN Network
 ip address 192.168.10.1 255.255.255.0
 ip access-group BLOCK-ICMP-PCB in
 duplex auto
 speed auto
!
interface GigabitEthernet0/1
 description Gateway for LAN Network
 ip address 10.10.10.1 255.255.255.0
 duplex auto
 speed auto
!
interface Vlan1
 no ip address
 shutdown
!
ip classless
!
ip flow-export version 9
!
!
ip access-list extended BLOCK-ICMP-PCB
 deny icmp host 192.168.10.20 any
 permit ip any any
!
!
```

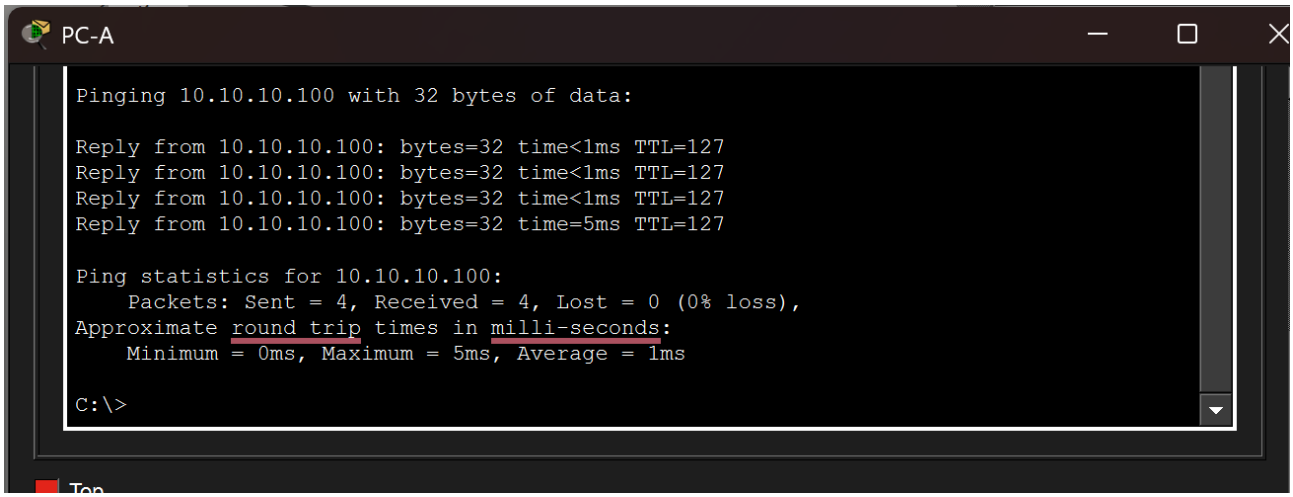
Phase 4 — Verify ACL Behavior (After ACL)

☑ Phase 4 Checkpoint

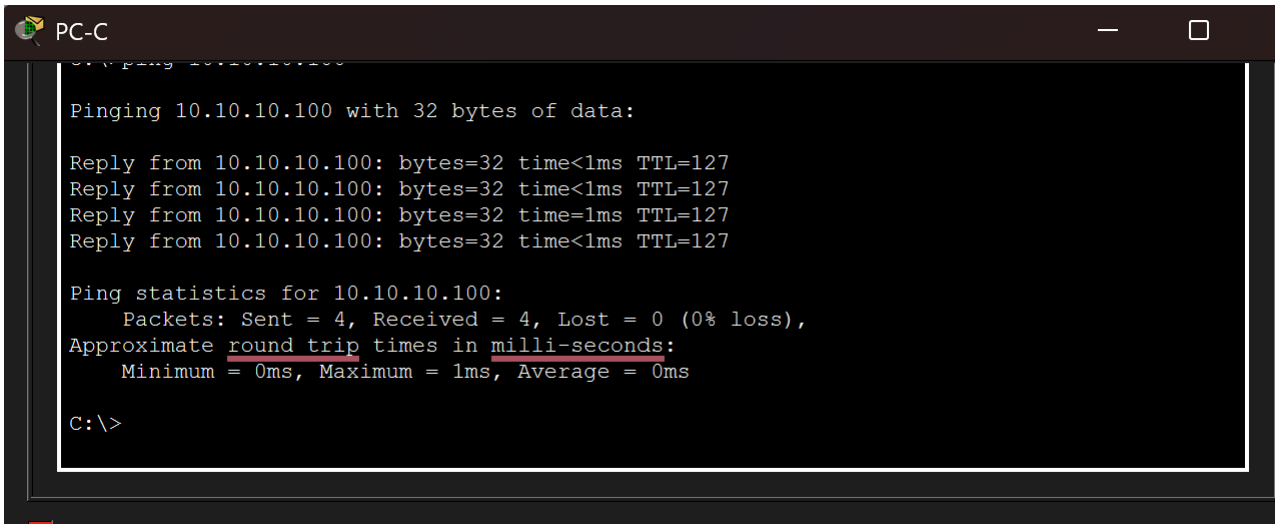
Ping from PC-B to WEB-SERVER — failed (request timed out).



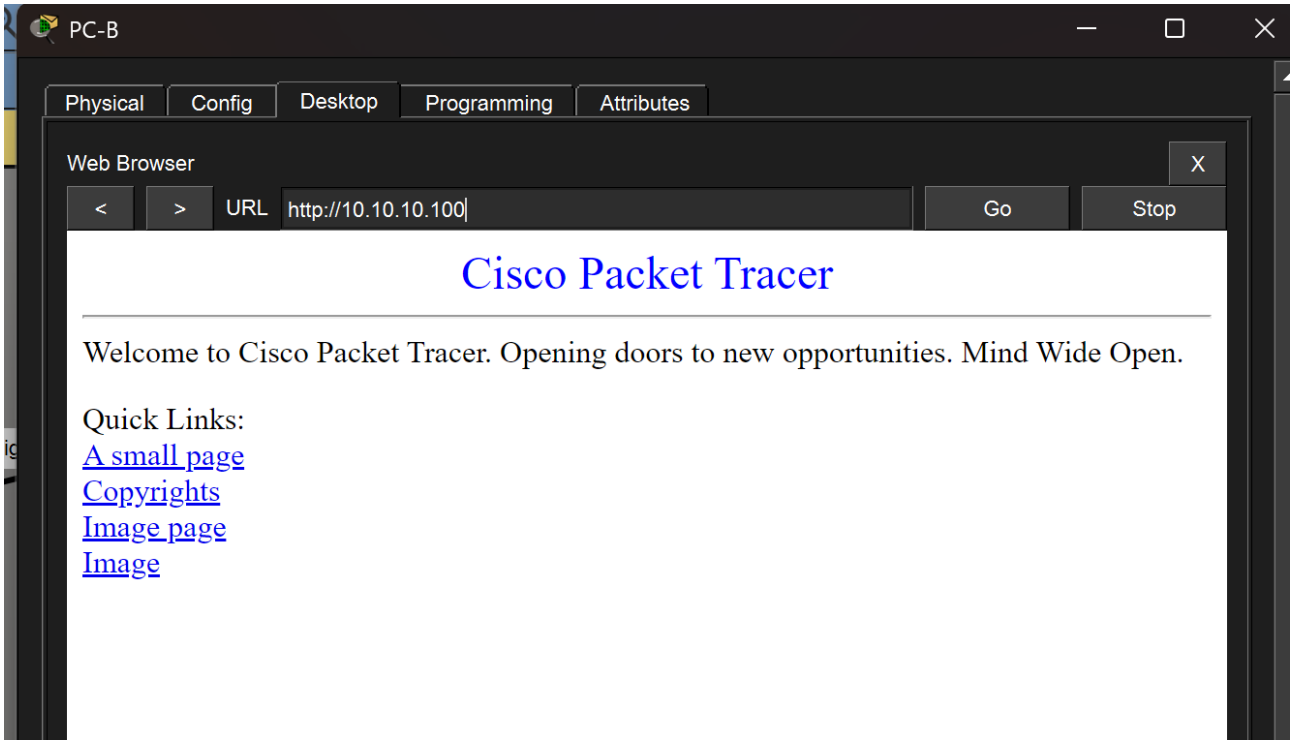
Ping from PC-A to WEB-SERVER — still successful (ACL does not affect PC-A).



Ping from PC-C to WEB-SERVER — still successful.



HTTP page loading in PC-B's browser from 10.10.10.100 (TCP still works).

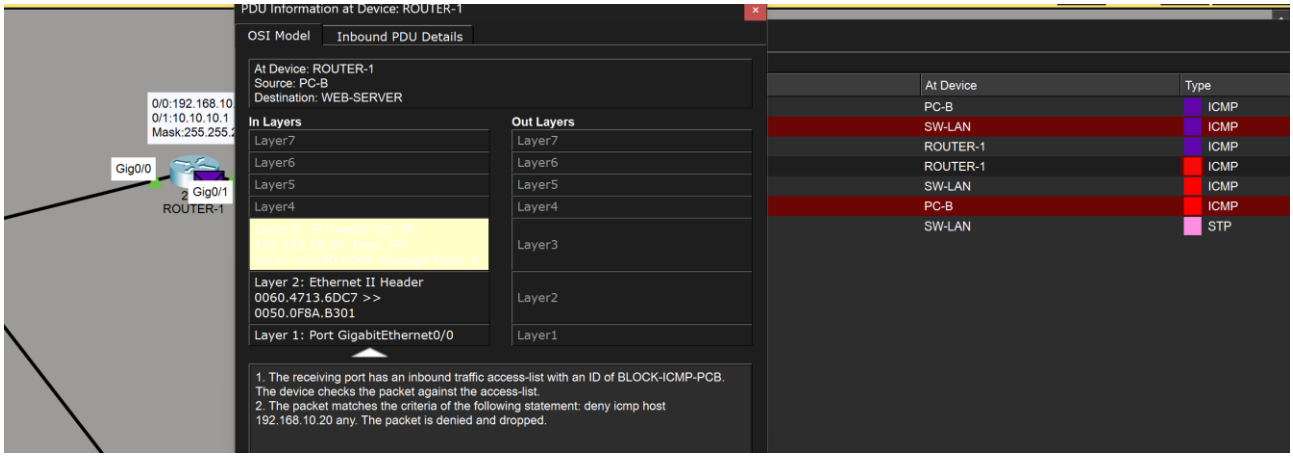


'show access-lists' output showing match counter on the deny rule (X matches).

```
ROUTER-1#show access-lists
Extended IP access list BLOCK-ICMP-PCB
    10 deny icmp host 192.168.10.20 any (4 match(es))
    20 permit ip any any (13 match(es))

ROUTER-1#
```

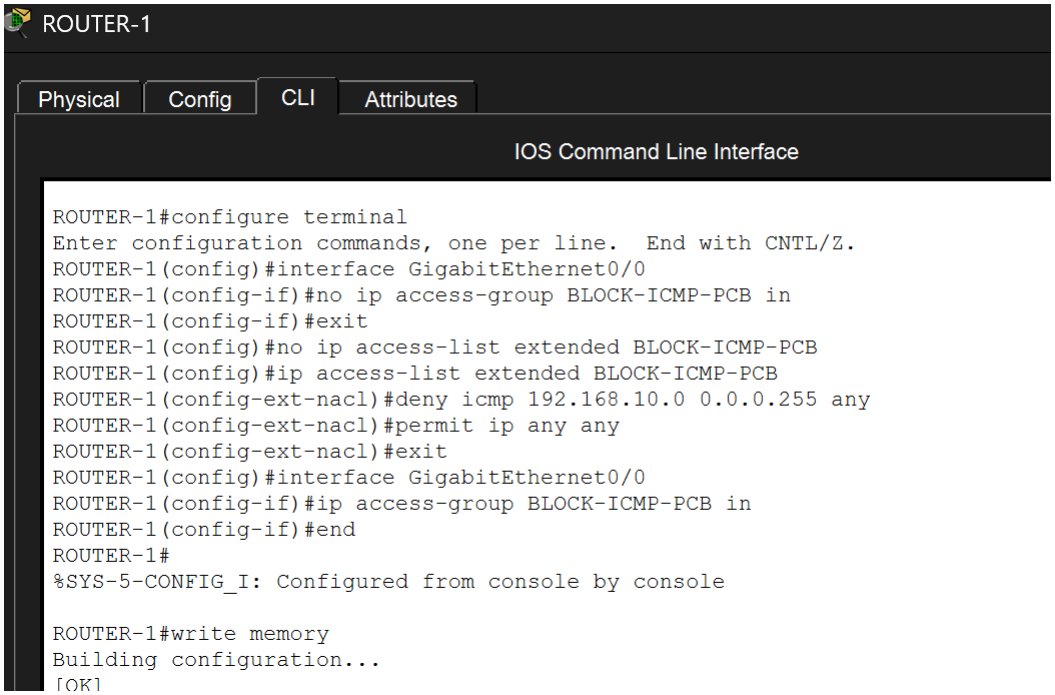
Simulation Mode PDU capture showing ICMP packet dropped at ROUTER-1.



Phase 5 — Challenge Task

Challenge: Extend the ACL to Block ICMP from a Subnet Range

Implemetation:



Prove:

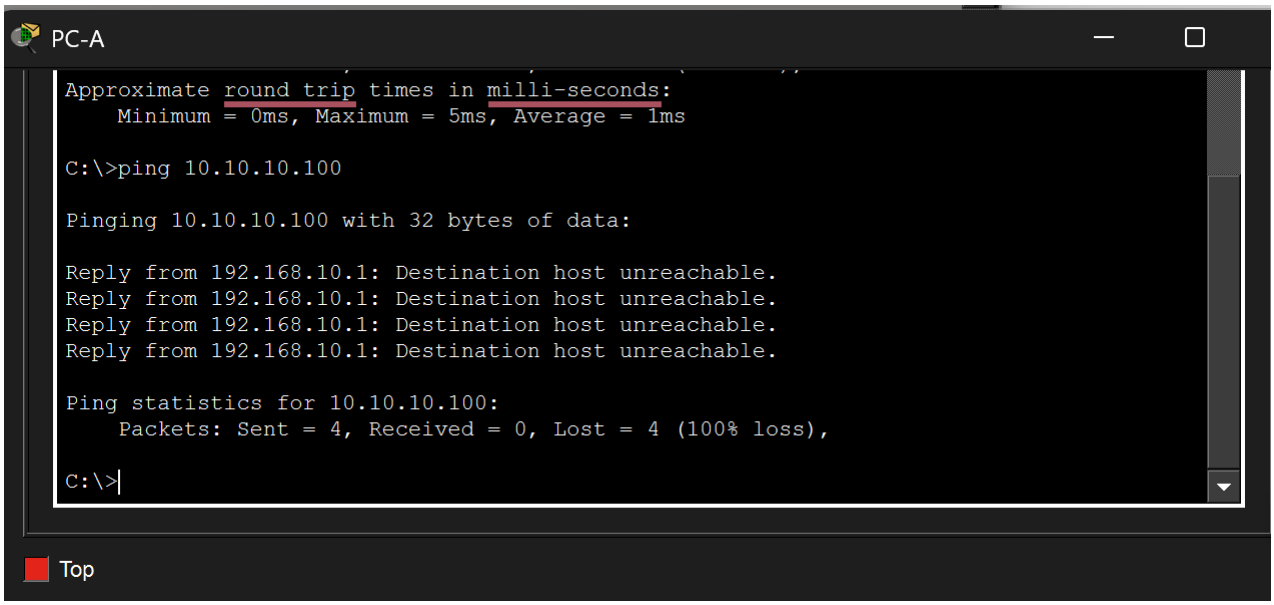
```
[OK]
ROUTER-1#show ip interface GigabitEthernet0/0
GigabitEthernet0/0 is up, line protocol is up (connected)
  Internet address is 192.168.10.1/24
  Broadcast address is 255.255.255.255
  Address determined by setup command
  MTU is 1500 bytes
  Helper address is not set
  Directed broadcast forwarding is disabled
  Outgoing access list is not set
  Inbound  access list is BLOCK-ICMP-PCB
  Proxy ARP is enabled
  Security level is default
  Split horizon is enabled
  ICMP redirects are always sent
  ICMP unreachables are always sent
  ICMP mask replies are never sent
  IP fast switching is disabled
  IP fast switching on the same interface is disabled
  IP Flow switching is disabled
  IP Fast switching turbo vector
  IP multicast fast switching is disabled
  IP multicast distributed fast switching is disabled
  Router Discovery is disabled
  IP output packet accounting is disabled
  IP access violation accounting is disabled
  TCP/IP header compression is disabled
  RTP/IP header compression is disabled
  Probe proxy name replies are disabled
  Policy routing is disabled
  Network address translation is disabled
  BGP Policy Mapping is disabled
  Input features: MCI Check
  WCCP Redirect outbound is disabled
  WCCP Redirect inbound is disabled
  WCCP Redirect exclude is disabled
```

Access List:

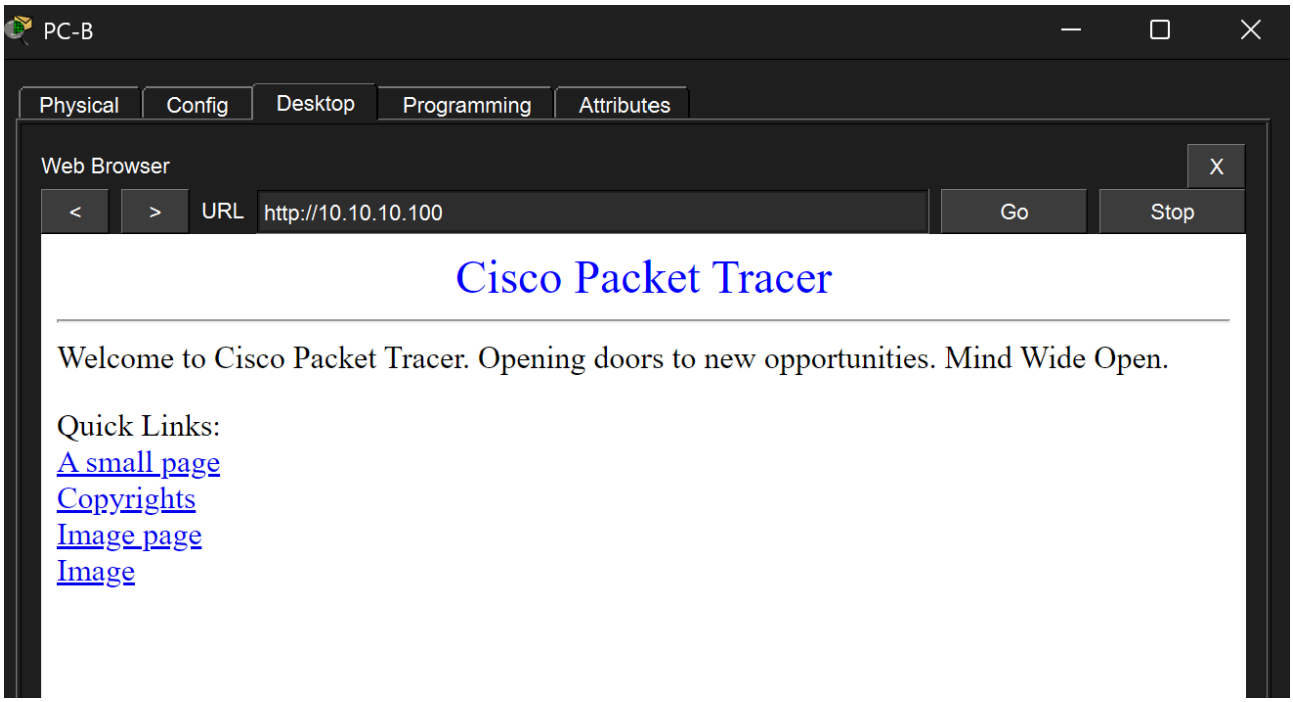
```
ROUTER-1#
ROUTER-1#show access-lists
Extended IP access list BLOCK-ICMP-PCB
    10 deny icmp 192.168.10.0 0.0.0.255 any (4 match(es))
    20 permit ip any any (5 match(es))

ROUTER-1#
```

Ping Failed:



Web Browser:



Matching a Single Host

- **Syntax Example**: deny icmp host 192.168.10.20 any
- **How it Works**: The host keyword tells the router to look for one exact IP address
- **When to Use**: When you need to block or allow one specific machine (like a single "blocked" PC-B) without affecting any other devices on that same network.

Matching a Subnet

- **Syntax Example**: deny icmp 192.168.10.0 0.0.0.255 any
- **How it Works**: Instead of a specific IP, you provide a network address followed by a Wildcard Mask.
    - A wildcard mask is the inverse of a subnet mask.
    - **0** means the octet **must match** exactly.
    - **255** means the router **ignores** that octet (it can be any number from 0-255).
- **When to Use**: When you want to apply a policy to an entire department or floor.
- **Efficiency**: It is much faster and cleaner to write one line for a subnet than to write 254 individual lines for every host.

1. Objective

The main goal of this lab was to learn how to control network traffic using a "Named Extended ACL" on a router. In the real world, filtering traffic like this is important because it lets a network admin block "ping" tests that hackers use to find active devices. It also helps stop certain attacks that try to crash a server by flooding it with too many messages, all without blocking the actual website or services people need to use.

2. Topology Summary

The design consisted of two distinct network segments: a Local Area Network (LAN) using the 192.168.10.0/24 range and a Server Network using the 10.10.10.0/24 range. These were separated by **ROUTER-1**, which acted as the gateway between the users and the sensitive server data. It ensures that traffic must pass through a Layer 3 device (the router) where security policies can be strictly enforced.

3. ACL Design Decisions

A **named extended ACL** (BLOCK-ICMP-PCB) was chosen over a standard ACL because standard ACLs lack protocol awareness. They can only filter by Source IP. If a standard ACL had been used, blocking PC-B would have cut off its web access as well. The extended ACL allowed for accurate, matching specifically on the ICMP protocol while ignoring TCP traffic.

The ACL was applied **inbound on GigabitEthernet0/0**. This is the most efficient placement because the router evaluates and drops the unauthorized packets the moment they arrive, before wasting CPU cycles on a routing table lookup. If the permit ip any any statement had been omitted, the **Implicit Deny** at the end of the ACL would have blocked every device on the LAN from reaching the server for any reason.

4. Results & Evidence

Prior to the ACL implementation, all hosts (PC-A, PC-B, and PC-C) had full connectivity to the WEB-SERVER, as demonstrated in Screenshot 9, which shows a successful ICMP PDU traversing the router. After applying the ACL, PC-B was successfully restricted. As seen in Screenshot 13, the ping from PC-B resulted in a "Request timed out," while Screenshot 16 proves that PC-B could still access the web interface via HTTP.

The router's active enforcement was confirmed using the show access-lists command, where the deny icmp line showed 4 matches, corresponding to the four packets sent during the failed ping attempt.

5. Connection to DoS Defense

This lab shows a basic way to defend against **Denial of Service (DoS) attacks**. In these attacks, someone tries to overwhelm a server with so much junk traffic that it crashes. By using an ACL to block or limit ICMP traffic, we can stop "Ping Floods" from taking down our servers. However, a simple ACL isn't enough to stop a massive attack from thousands of different computers at once. In a real company, we would need extra tools to make sure the network stays up during a big attack.

6. Conclusion

In conclusion, I successfully reached the goal of blocking pings from one specific computer while letting all other traffic through. The lab showed me exactly how much control a router has over a network. Overall, the lab was a great way to see how security rules work in a real-life scenario.